
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION COLD IRON - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 20, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-I-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-I.bin          b6fced1c0c305fdf167ed2da93464997504e1126d552eb120016d34c3a0bc47a
ACT-I.uf2          015e31dfeec6728afbc3a606d8b7b0283666f994acf124e56d0ed182119033f4
ACT-I_fixed.bin    e63c513a8b798c0e4be9265e4313b8d882f3b6fd8e9ae1161316adad7a926e29
ACT-I_fixed.uf2    f5df73b6ca69d4384ae64ffc39920e8d36634639a12b23904a962bd743332bf8
```

Machine check: `python scripts/verify_ctf.py` returns 12/12 checks passed against the shipped and corrected images. It asserts the six byte pairs and the `ACT-I.bin` and `ACT-I_fixed.bin` SHA-256 values.

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-I.bin` as `Raw Binary`, language `ARM Cortex 32 little endian default`, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `ColdIron_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-I.bin`:

```
00 20 08 20 5B 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	<code>0x10000000</code>
Initial SP	<code>0x20082000</code>
Reset handler (as stored)	<code>0x1000015B</code>
Reset instruction address	<code>0x1000015A</code>

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry `0x1000015A`.

Entry and Monitor Loop. From `ACT-I-main-disasm.txt`:

```

100001e0 <main>:
100001e0: b508      push    {r3, lr}
100001e2: f003 faaf  bl     10003744 <stdio_init_all>
100001e6: 4807      ldr     r0, [pc, #28]
100001e8: f003 faf6  bl     100037d8 <__wrap_puts>
100001ec: f006 f8e6  bl     100063bc <monitor_init>
100001f0: b110      cbz     r0, 100001f8 <main+0x18>
100001f2: f006 f991  bl     10006518 <monitor_step>
100001f6: e7fc      b.n     100001f2 <main+0x12>

```

Element	Address
main	0x100001E0
monitor_init	0x100063BC
monitor_step	0x10006518

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Random source	get_rand_32	0x1000631C
KDF	crypto_kdf_argon2id	0x100077F0
Sensor	sensor_read	0x10006A4C
Display	display_format_lines	0x10006ED8
Radio	radio_send_frame	0x10007254
Status LEDs	status_led_init	0x10007504
Status LEDs	status_led_show	0x1000754C
Status LEDs	status_led_state_for_temperature	0x1000759C
Button	button_init	0x100075AC
Button	button_consume_press	0x100075C8
Servo	servo_init	0x10007630
Servo	servo_seal	0x1000767C
Servo	servo_vent	0x10007698
Infrared	ir_remote_init	0x100076B8
Infrared	ir_remote_poll	0x10007758
Envelope	envelope_fill_nonce	0x10007858
Envelope	envelope_seal_hex	0x10007880
Envelope	envelope_open_hex	0x10007940
Crypto	crypto_aead_open	0x10007A80

The six sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Cold offset	status_led_state_for_temperature	0x759E	0x1000759E	0xFA	0x00
2 Dead annunciator	status_led_show	0x754F	0x1000754F	0xD1	0xD0
3 Damper geometry	servo_seal	0x7686	0x10007686	0x48	0xFA
4 Open infrared door	inlined_monitor_ir_command in monitor_step	0x6569	0x10006569	0xD0	0xD1
5 Constant tag	crypto_aead_open	0x7AFF	0x10007AFF	0xD1	0xD0
6 Nonce reuse	envelope_fill_nonce	0x7858	0x10007858	0x00	0x18

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>ColdIron_Investigation</code> , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015B</code>
[DOCUMENT] main and the monitor loop (monitor_step) addresses identified	1	main <code>0x100001E0</code> , monitor_step <code>0x10006518</code>
[DOCUMENT] Module map identifies the sensor, display, radio, status LEDs, button, servo, infrared, envelope, crypto, and KDF anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary`, `ARM Cortex 32 little endian default`, base `0x10000000`, and that auto-analysis completed before any address was read.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015B` is odd because bit 0 selects Thumb; clearing it gives `0x1000015A`.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient.
- The six-site summary table is the reference map students should build toward; do not award Task 1 credit for it unless the anchor addresses rows are also supported.

Task 2: Bug #1 Cold Offset (12 points)

Solution

Locate the compare. In `status_led_state_for_temperature` the breach threshold compare is at file offset `0x759E` (VA `0x1000759E`). The corrected image (from `ACT-I-main-disasm.txt`) is:

```
1000759c <status_led_state_for_temperature>:
1000759c: b121      cbz    r1, 100075a8 <status_led_state_for_temperature+0xc>
1000759e: 2800      cmp    r0, #0
100075a0: bfac      ite    ge
100075a2: 2003      movge  r0, #3
100075a4: 2001      movlt  r0, #1
100075a6: 4770      bx     lr
100075a8: 4608      mov    r0, r1
100075aa: 4770      bx     lr
```

Instruction decode.

Address	File offset	Compromised bytes	Compromised instruction	Correct bytes	Correct instruction
<code>0x1000759E</code>	<code>0x759E</code>	<code>FA 28</code>	<code>cmp r0, #250</code>	<code>00 28</code>	<code>cmp r0, #0</code>

The compare is against `COLD_CHAIN_MONITOR_TEMP_BREACH_TENTHS`, the breach ceiling in tenths of a degree Celsius. The correct constant is `0` (0.0 C). The compromised immediate `0xFA` is `250`, which is 25.0 C, far above the warm reading, so the `ite ge / movge` path selects `STATUS_LED_NOMINAL`.

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x759E</code>	<code>0x1000759E</code>	<code>FA 28</code>	<code>00 28</code>

Why a warm reading now alarms. A warm store reads 5.0 C, which is 50 tenths. Under the compromise, `cmp r0, #250` tests `50 >= 250`, which is false, so the function returns `STATUS_LED_NOMINAL` (1). After the patch, `cmp r0, #0` tests `50 >= 0`, which is true, so the function returns `STATUS_LED_BREACH` (3) and the breach path runs.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the cold offset compare at <code>0x1000759E</code>	3	Address and function identified
[DOCUMENT] Documented the compromised immediate and the correct threshold	3	Compromised <code>0xFA</code> (250), correct <code>0x00</code> (0)
[DOCUMENT & PATCH] Patched <code>0xFA</code> to <code>0x00</code> so a warm reading classifies as a breach	4	Byte <code>0xFA</code> changed to <code>0x00</code>
[DOCUMENT] Explained that a 5.0 C reading (50 tenths) now alarms	2	50 tenths at or above 0.0 C, so BREACH

Instructor Notes & Assembly

- The byte is the low byte of the 16-bit Thumb compare `2800 (cmp r0, #0)`.
- The compromised value `0xFA` is `250`, not `250.0 C`; the unit is tenths.
- Accept any correct demonstration that `50 >= 250` is false and `50 >= 0` is true. Full credit requires both the address and both bytes.
- The correct constant is the zero degree Celsius breach ceiling.

Task 3: Bug #2 Dead Annunciator (12 points)

Solution

Locate the branch. In `status_led_show` the breach branch is at file offset `0x754F` (VA `0x1000754F`). The corrected image is:

```

1000754c <status_led_show>:
1000754c: 2803      cmp    r0, #3
1000754e: d016      beq.n 1000757e <status_led_show+0x32>
10007550: f010 0ffd  tst.w  r0, #253
10007554: f04f 0312  mov.w  r3, #18
10007558: bf14      ite   ne
1000755a: 2201      movne r2, #1
1000755c: 2200      moveq  r2, #0
1000755e: ec42 3040  mcrr   0, 4, r3, r2, cr0
10007562: f1a0 0002  sub.w  r0, r0, #2
10007566: fab0 f080  clz    r0, r0
1000756a: 2311      movs   r3, #17
1000756c: 0940      lsrs   r0, r0, #5
1000756e: ec40 3040  mcrr   0, 4, r3, r0, cr0
10007572: 2310      movs   r3, #16
10007574: f04f 0200  mov.w  r2, #0
10007578: ec42 3040  mcrr   0, 4, r3, r2, cr0
1000757c: 4770      bx     lr
1000757e: 2212      movs   r2, #18
10007580: f04f 0300  mov.w  r3, #0
10007584: ec43 2040  mcrr   0, 4, r2, r3, cr0
10007588: 2211      movs   r2, #17
1000758a: ec43 2040  mcrr   0, 4, r2, r3, cr0
1000758e: 2310      movs   r3, #16
10007590: f04f 0201  mov.w  r2, #1
10007594: ec42 3040  mcrr   0, 4, r3, r2, cr0
10007598: 4770      bx     lr

```

Instruction decode. The `cmp r0, #3` at `0x1000754C` tests for `STATUS_LED_BREACH`. When the state is breach, the correct branch target is `0x1000757E`, the red lamp path (green `0x757E` off, yellow off, red on at `0x10007590`). The branch condition byte at `0x754F` therefore must be `0xD0` (`beq.n`, taken on equality).

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000754F</code>	<code>0x754F</code>	<code>0xD1</code>	<code>bne.n 0x1000757E</code>	<code>0xD0</code>	<code>beq.n 0x1000757E</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x754F</code>	<code>0x1000754F</code>	<code>16 D1</code>	<code>16 D0</code>

Why the red lamp now lights. Under the compromised `bne.n`, the branch is taken whenever the state is not breach, so a breach falls through into the green nominal path at `0x10007550` and the red lamp is never driven. After the patch, `beq.n` sends the breach state to `0x1000757E`, which drives green off, yellow off, and red on.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the annunciator branch at 0x1000754F	3	Address and function identified
[DOCUMENT] Documented <code>beq.n</code> versus <code>bne.n</code> and the branch target	3	Correct branch and target <code>0x1000757E</code>
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so a breach routes to the red lamp	4	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained why the compromised branch leaves the HMI looking healthy	2	Breach falls into the green nominal path

Instructor Notes & Assembly

- The condition byte is at `0x754F`; the instruction halfword is `16 D0` for `beq.n` and `16 D1` for `bne.n`.
- The red-lamp path is `0x1000757E`; it turns green and yellow off and red on.
- A common error is reversing the explanation. Under the compromise a breach is the one state that does not branch, so it reaches the green code.
- Full credit requires the byte change and a correct statement of which state reaches which path.

Task 4: Bug #3 Damper Geometry (12 points)

Solution

Locate the pulse constant. In `servo_seal` the seal pulse is built at file offset `0x7686` (VA `0x10007686`). The corrected image is:

```

1000767c <servo_seal>:
1000767c: 4b04      ldr    r3, [pc, #16]    @ 0x10007690 -> 0x400a8000
1000767e: 4a05      ldr    r2, [pc, #20]    @ 0x10007694 -> 0x400a9000
10007680: f8d3 3098 ldr.w  r3, [r3, #152]
10007684: f483 73fa eor.w  r3, r3, #500 @ 0x1f4
10007688: b29b      uxth   r3, r3
1000768a: f8c2 3098 str.w  r3, [r2, #152]
1000768e: 4770      bx     lr

```

Instruction decode. The modified immediate is the low byte of the second halfword. The correct byte `0xFA` encodes `500` microseconds, the zero-degree seal pulse. The compromised byte `0x48` encodes `800` microseconds.

Address	File offset	Compromised bytes	Compromised immediate	Correct bytes	Correct immediate
<code>0x10007686</code>	<code>0x7686</code>	<code>83 F4 48 73</code>	<code>#800</code>	<code>83 F4 FA 73</code>	<code>#500</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0x7686	0x10007686	48	FA

Geometry. The servo maps 500 us to 0 degrees and 2500 us to 180 degrees, so each degree is $2000/180 = 11.11$ us. A pulse of 500 us is the sealed angle. The compromised 800 us pulse is $(800 - 500) / 11.11 = 27.0$ degrees, so the damper stops about 27 degrees short of sealing and the cold store never fully closes.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the damper seal pulse constant at 0x10007686	3	Address and function identified
[DOCUMENT] Documented the 800 versus 500 microsecond pulse and its angle	3	Compromised 800 us near 27 degrees, correct 500 us
[DOCUMENT & PATCH] Patched 0x48 to 0xFA so the damper seals	4	Byte 0x48 changed to 0xFA
[DOCUMENT] Explained why the geometry error is a physical failure	2	The damper never fully closes so the store warms

Instructor Notes & Assembly

- The immediate is a modified-immediate field, so only the low byte changes: 0x48 to 0xFA.
- The mapping is 500 us seal (0 degrees) and 2500 us vent (180 degrees), so 800 us is about 27 degrees.
- Accept any correct angle derivation within one degree.
- The physical point is that the damper never seals; the store warms even when the lamps and the classifier are fixed.

Task 5: Bug #4 Open Infrared Door (12 points)

Solution

Locate the gate. The infrared command handler is inlined into `monitor_step`. The address gate is at file offset 0x6569 (VA 0x10006569). The corrected image is:

```

10006562: f89d 306d   ldrb.w r3, [sp, #109]   @ remote address
10006566: 2b1d        cmp    r3, #29          @ 0x1d authorized remote
10006568: d1dc        bne.n 10006524          @ reject non-maintenance address
1000656a: f89d 306e   ldrb.w r3, [sp, #110]   @ command
1000656e: 2b47        cmp    r3, #71          @ 0x47 VENT
10006570: f000 8093   beq.w 1000669a          @ servo_vent
10006574: 2b45        cmp    r3, #69          @ 0x45 SEAL
10006576: d1d5        bne.n 10006524          @ servo_seal
10006578: f001 f880   bl     1000767c          @ servo_seal

```

Instruction decode. The authorized maintenance remote address is `MONITOR_IR_REMOTE_ADDRESS = 0x1D (29)`. The correct code rejects any address that does not match, so the branch at 0x10006568 must be `bne.n (0xD1)`: branch back to the caller when `address != 29`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x10006569	0x6569	0xD0	beq.n 0x10006524	0xD1	bne.n 0x10006524

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0x6569	0x10006569	DC D0	DC D1

Why every address passes under the compromise. The compromised `beq.n` branches back to the caller only when `address == 29`. Every other address falls through to the command decode and can move the damper, so any NEC remote is effectively authorized. After the patch, `bne.n` rejects every address that is not `0x1D`, and only the maintenance remote can vent or seal.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the infrared address gate at 0x10006569	3	Address and inlined handler identified
[DOCUMENT] Documented the authorized address and the inverted branch	3	Address <code>0x1D</code> (29), compromised <code>beq.n</code>
[DOCUMENT & PATCH] Patched 0xD0 to 0xD1 so only the maintenance address is accepted	4	Byte <code>0xD0</code> changed to <code>0xD1</code>
[DOCUMENT] Explained why the compromised gate accepts every address	2	The reject branch is taken only on a match

Instructor Notes & Assembly

- The handler is inlined, so the students must read inside `monitor_step` rather than a standalone `monitor_ir_command` symbol.
- The authorized address constant is `0x1D` (29), matching `MONITOR_IR_REMOTE_ADDRESS`.
- The compromised `beq.n` means the reject path is taken only on a match, so non-matching addresses pass. Full credit requires that inversion to be stated correctly.
- Fixed firmware accepts only address `0x1D` and still requires the VENT or SEAL command byte.

Task 6: Bug #5 Constant Tag (12 points)**Solution**

Locate the branch. In `crypto_aead_open` the tag comparison folds the four 32-bit XOR words with OR into `r3`, then masks and tests the low byte. The branch is at file offset `0x7AFF` (VA `0x10007AFF`). The corrected image is:

```

10007ac8: f001 fca2    bl     10009410 <poly1305_mac_aead>
10007acc: 9a08        ldr     r2, [sp, #32]
10007ace: 6833        ldr     r3, [r6, #0]
10007ad0: 9909        ldr     r1, [sp, #36]
10007ad2: 4053        eors    r3, r2
10007ad4: 6872        ldr     r2, [r6, #4]
10007ad6: 980a        ldr     r0, [sp, #40]
10007ad8: 404a        eors    r2, r1
10007ada: 68b1        ldr     r1, [r6, #8]
10007adc: 4313        orrs    r3, r2
10007ade: 4041        eors    r1, r0
10007ae0: 430b        orrs    r3, r1
10007ae2: 68f2        ldr     r2, [r6, #12]
10007ae4: 990b        ldr     r1, [sp, #44]
10007ae6: 404a        eors    r2, r1
10007ae8: 4313        orrs    r3, r2
10007aea: f3c3 2207    ubfx    r2, r3, #8, #8
10007aee: 431a        orrs    r2, r3
10007af0: f3c3 4107    ubfx    r1, r3, #16, #8
10007af4: 430a        orrs    r2, r1
10007af6: ea42 6313    orr.w   r3, r2, r3, lsr #24
10007afa: f013 03ff    ands.w  r3, r3, #255
10007afe: d003        beq.n   10007b08 <crypto_aead_open+0x88>
10007b00: 4620        mov     r0, r4
10007b02: b025        add     sp, #148
10007b04: e8bd 83f0    ldmia.w sp!, {r4, r5, r6, r7, r8, r9, pc}
10007b08: ...        decrypt and return 1

```

Instruction decode. The compare is a constant-time tag check: the computed tag is XORed against the received tag word by word, the differences are ORed together, and the result is masked. A zero result means the tags match. The correct code accepts only a zero difference, so the branch at `0x10007AFE` must be `beq.n (0xD0)` to the accept path at `0x10007B08`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10007AFF</code>	<code>0x7AFF</code>	<code>0xD1</code>	<code>bne.n 0x10007B08</code>	<code>0xD0</code>	<code>beq.n 0x10007B08</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x7AFF</code>	<code>0x10007AFF</code>	<code>03 D1</code>	<code>03 D0</code>

Why forged frames now fail. Under the compromised `bne.n`, a non-zero tag difference branches to the accept path, so a forged frame is decrypted and accepted while a genuine frame with a zero difference falls through and is rejected. After the patch, only a zero difference reaches `0x10007B08`, so a forged frame returns 0 and the gateway logs `UNAUTHENTICATED`.

The constant-time construction never returns early on the first differing byte; it always ORs all word differences, so timing does not reveal how many bytes matched.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the tag-difference branch at <code>0x10007AFF</code>	3	Address and function identified

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Documented the constant-time compare and the branch condition	3	Zero difference means accept
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so forged frames fail authentication	4	Byte 0xD1 changed to 0xD0
[DOCUMENT] Explained why a non-zero tag difference must be rejected	2	A non-zero difference means forged or corrupt

Instructor Notes & Assembly

- The branch condition byte is at 0x7AFF ; the halfword is 03 D0 for beq.n and 03 D1 for bne.n .
- The compare is a full word-wise OR of the four tag-difference words, folded into a single zero test at 0x10007AFA .
- Full credit requires the inversion explanation: the compromised build accepts a non-zero difference and rejects a zero difference.
- Point out that the rest of the AEAD is correct; only this seam was broken.

Task 7: Bug #6 Nonce Reuse (12 points)

Solution

Locate the loop bound. In `envelope_fill_nonce` the loop bound is at file offset 0x7858 (VA 0x10007858). The corrected image is:

```

10007858 <envelope_fill_nonce>:
10007858: 2318      movs r3, #24
1000785a: b510      push {r4, lr}
1000785c: b082      sub sp, #8
1000785e: 9301      str r3, [sp, #4]
10007860: 9b01      ldr r3, [sp, #4]
10007862: b153      cbz r3, 1000787a <envelope_fill_nonce+0x22>
10007864: 4604      mov r4, r0
10007866: f7fe fd59 bl 1000631c <get_rand_32>
1000786a: 9b01      ldr r3, [sp, #4]
1000786c: f844 0b04 str.w r0, [r4], #4
10007870: 3b04      subs r3, #4
10007872: 9301      str r3, [sp, #4]
10007874: 9b01      ldr r3, [sp, #4]
10007876: 2b00      cmp r3, #0
10007878: d1f5      bne.n 10007866 <envelope_fill_nonce+0xe>
1000787a: b002      add sp, #8
1000787c: bd10      pop {r4, pc}

```

Instruction decode. The loop bound is the immediate 24, the nonce length in bytes. The loop calls `get_rand_32()` once per four bytes and stores the word little-endian, decrementing the bound by four each pass.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x10007858	0x7858	0x00	movs r3, #0	0x18	movs r3, #24

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0x7858	0x10007858	00 23	18 23

Why the nonce is reused under the compromise. With `movs r3, #0`, the `cbz r3, 0x1000787A` at `0x10007862` jumps straight to the function exit. The loop body never runs, so `get_rand_32()` is never called and the 24-byte nonce buffer keeps whatever the caller's stack held. Every sealed frame therefore uses the same nonce. With XChaCha20-Poly1305, reusing a nonce under the same key turns the keystream into a repeated keystream, and XORing two ciphertexts cancels the keystream to reveal the XOR of the plaintexts. After the patch the loop runs six times, drawing 24 fresh bytes per frame, so each frame has a unique nonce.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the nonce fill loop bound at 0x10007858	3	Address and function identified
[DOCUMENT] Documented the compromised and correct loop bounds	3	Compromised <code>0x00</code> , correct <code>0x18</code> (24)
[DOCUMENT & PATCH] Patched 0x00 to 0x18 so 24 random bytes are drawn	4	Byte <code>0x00</code> changed to <code>0x18</code>
[DOCUMENT] Explained nonce reuse and how a fresh random nonce stops it	2	Zero iterations means a fixed nonce; 24 random bytes fixes it

Instructor Notes & Assembly

- The bound is the immediate at `0x7858`; the halfword is `18 23` for `movs r3, #24` and `00 23` for `movs r3, #0`.
- The loop calls `get_rand_32()` at `0x10007866`; the zero bound skips the `cbz` fall-through and exits at `0x1000787A`.
- The nonce is 24 bytes and the loop steps by four, so the fixed bound draws six random words.
- Full credit requires both the byte change and an explanation of the reused keystream or the fixed nonce.

Task 8: Runtime Key Capture (8 points)

Solution

Breakpoint after the KDF. In `monitor_init` the field key is derived by `crypto_kdf_argon2id` at `0x100064CA`, and the call returns to `0x100064CE`. Break at the return address and read the 32-byte key from SRAM at `0x20013074` (`g_key`).

```
arm-none-eabi-gdb ACT-I.elf
(gdb) target extended-remote localhost:3333
(gdb) monitor reset halt
(gdb) break *0x100064CE
(gdb) continue
(gdb) x/32xb 0x20013074
(gdb) x/8xb 0x20013074
```

Element	Address
crypto_kdf_argon2id call	0x100064CA
Return address (breakpoint)	0x100064CE
Derived key buffer g_key	0x20013074
Key length	32 bytes

Expected relationship. The captured 32 bytes are the Argon2id output of the provisioned passphrase and salt. The capture is correct when the 32 bytes read from 0x20013074 equal the key the Python gateway derives from the same passphrase and salt (scripts/field_crypto.py). Do not record a guessed value: the accepted evidence is the GDB transcript plus a matching offline derivation. The exact hex is a deterministic function of the passphrase and salt recovered in Task 9.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Breakpoint set at 0x100064CE after the crypto_kdf_argon2id call	3	Correct breakpoint address and command sequence
[DOCUMENT] Captured the 32-byte key from SRAM 0x20013074	3	32-byte hex read from g_key
[DOCUMENT] Compared the captured key to the gateway Argon2id derivation	2	Captured key equals the derived key

Instructor Notes & Assembly

- Confirm the breakpoint is placed at 0x100064CE, the instruction after the bl crypto_kdf_argon2id at 0x100064CA in monitor_init.
- The key lives in SRAM, not flash, because g_key is a writable runtime buffer.
- Do not accept a zeroed buffer or a copied constant: a correct capture must match the independent Argon2id derivation from Task 9.
- The task explicitly forbids fabricating the key; credit the process and the match, not a memorized hex.

Task 9: Provisioning Recovery (5 points)

Solution

Recover the passphrase and salt.

Secret	Value	Flash location
Passphrase	operation cold iron field key v1	.rodata, seen in Ghidra Defined Strings
Salt	coldiron-salt-01	.rodata, 16 bytes

The passphrase is the ASCII string operation cold iron field key v1. The salt is the 16 ASCII bytes of coldiron-salt-01, found in flash as:

```
63 6F 6C 64 69 72 6F 6E 2D 73 61 6C 74 2D 30 31
c o l d i r o n - s a l t - 0 1
```

Use the Ghidra **Defined Strings** view to jump to the passphrase, or search the raw image for the salt byte pattern above. Record the address of both.

Production requirement. These values are committed in flash as a lab convenience so the firmware and the instructor gateway derive the same session key. A committed passphrase and salt mean anyone with the image can derive the field key. Production firmware must provision the session key from one-time-programmable (OTP) memory at manufacture and must never embed a passphrase or a derived key in flash.

Offline re-derivation. Re-derive the 32-byte key with Argon2id using time cost 3, parallelism 1, and memory 64 KiB (64 blocks), over the passphrase and the 16-byte salt. A correct re-derivation equals the Task 8 capture byte for byte.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Recovered the passphrase and salt from flash	2	operation cold iron field key v1 and coldiron-salt-01
[DOCUMENT] Explained that committed secrets are dev-only and production must use OTP	1	OTP provisioning requirement stated
[DOCUMENT] Re-derived the key offline with Argon2id and matched the Task 8 capture	2	t=3, p=1, m=64 and a match

Instructor Notes & Assembly

- The passphrase is exactly operation cold iron field key v1 and the salt is exactly coldiron-salt-01 (16 bytes).
- Grade the OTP point on the production requirement: key material comes from OTP at manufacture, never from committed flash.
- The offline parameters are time cost 3, parallelism 1, memory 64 blocks (64 KiB), output 32 bytes.
- A correct Task 9 derivation must match the Task 8 capture; if it does not, the likely cause is a wrong salt length or a wrong Argon2 variant.

Task 10: Export and Verify (5 points)

Solution

Export. In Ghidra, File -> Export Program..., choose Binary Format, and save as ACT-I_fixed.bin. The shipped image is 48,596 bytes.

Convert.

```
python uf2conv.py ACT-I_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-I_fixed.uf2
```

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

12/12 checks passed

Hardware proof. Flash `ACT-I_fixed.uf2` in BOOTSEL mode and confirm:

- a warm reading lights the red lamp and vents the damper;
- the damper seals when the store recovers;
- a non-maintenance NEC remote no longer moves the damper;
- the gateway logs `UNAUTHENTICATED` for a forged envelope and authenticates a genuine frame with a fresh nonce.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	Cold offset	0x759E	0x1000759E	FA	00
2	Dead annunciator	0x754F	0x1000754F	D1	D0
3	Damper geometry	0x7686	0x10007686	48	FA
4	Open infrared door	0x6569	0x10006569	D0	D1
5	Constant tag	0x7AFF	0x10007AFF	D1	D0
6	Nonce reuse	0x7858	0x10007858	00	18

Reflection mapping. The six defects map to real cold-chain failures:

Defect	Real-world failure
Cold offset	A classifier that treats a warm reading as safe hides an excursion from the operator.
Dead annunciator	A status lamp that never lights fails the operator's final visual check.
Damper geometry	An actuator that never fully seals lets the store warm even when the logic is correct.
Open infrared door	An unauthenticated actuator port lets a stranger move the damper and deny the act.
Constant tag	A broken tag check lets forged telemetry look authentic to the gateway.
Nonce reuse	A repeated nonce leaks telemetry plaintext and breaks confidentiality.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported ACT-I_fixed.bin from Ghidra	1	Valid patched binary
[PATCH] Converted to ACT-I_fixed.uf2 with the correct base and family	1	<code>--base 0x10000000 --family 0xe48bff59</code>

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the red lamp, damper seal, and authentication	2	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the six defects to a real-world cold-chain failure	1	Specific mapping for all six

Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-I.bin` in exactly the six bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.
- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 48,596 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the six defects should name a concrete cold-chain consequence.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 data	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	220 to 330 ohm to GND
Yellow LED	anode	GP17	220 to 330 ohm to GND
Green LED	anode	GP18	220 to 330 ohm to GND
Button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Green blink on transmit
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 Cold Offset	12
Task 3	Bug #2 Dead Annunciator	12
Task 4	Bug #3 Damper Geometry	12
Task 5	Bug #4 Open Infrared Door	12
Task 6	Bug #5 Constant Tag	12
Task 7	Bug #6 Nonce Reuse	12
Task 8	Runtime Key Capture	8
Task 9	Provisioning Recovery	5
Task 10	Export and Verify	5
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational cold store, a pharmaceutical network, a public network, a military system, or a third-party device.

Common Student Mistakes

- Patching the high byte of the compare at `0x759E` instead of the low byte `0xFA`.
- Reversing the annunciator explanation: under the compromise a breach is the state that does not branch, so it reaches the green path.
- Treating the damper constant as a degree value; it is a pulse width, and `500` us is the seal.
- Searching for a standalone `monitor_ir_command` symbol and missing that it is inlined into `monitor_step`.
- Reading the tag branch backwards and believing the compromised build accepts genuine frames.
- Patching the `get_rand_32` call site instead of the loop bound at `0x7858`.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating the GDB-captured key instead of matching the Argon2id output.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct damper angle without the byte change, or the byte change without the geometry.
- Award partial credit for capturing the key without the gateway comparison, or the comparison without a clean capture.

- Award no credit for patches that alter any byte outside the six documented offsets, and no credit for a fabricated key.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```
--- ACT-I.bin (compromised)
+++ ACT-I_fixed.bin (corrected)

Offset 0x0000759E: FA -> 00 (cmp r0, #250 -> cmp r0, #0)
Offset 0x0000754F: D1 -> D0 (bne.n 0x1000757E -> beq.n 0x1000757E)
Offset 0x00007686: 48 -> FA (eor.w r3, r3, #800 -> eor.w r3, r3, #500)
Offset 0x00006569: D0 -> D1 (beq.n 0x10006524 -> bne.n 0x10006524)
Offset 0x00007AFF: D1 -> D0 (bne.n 0x10007B08 -> beq.n 0x10007B08)
Offset 0x00007858: 00 -> 18 (movs r3, #0 -> movs r3, #24)
```

#	Bug	File Offset(s)	Flash Address(es)	Original Bytes	Patched Bytes
1	Cold offset	0x759E	0x1000759E	FA 28	00 28
2	Dead annunciator	0x754F	0x1000754F	16 D1	16 D0
3	Damper geometry	0x7686	0x10007686	83 F4 48 73	83 F4 FA 73
4	Open infrared door	0x6569	0x10006569	DC D0	DC D1
5	Constant tag	0x7AFF	0x10007AFF	03 D1	03 D0
6	Nonce reuse	0x7858	0x10007858	00 23	18 23

Six defects, six changed bytes in six instructions: the cold-offset immediate, the annunciator condition, the damper pulse immediate, the infrared condition, the tag condition, and the nonce loop bound. No other byte in either image differs.